Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

The main purpose of this experiment was to understand how recurrent neural networks handle data where the order of observations matters. For this, Vanilla RNN, LSTM and GRU models were implemented and compared on a human activity recognition dataset.

Along with the basic recurrent models, the experiment was used to understand Backpropagation Through Time (BPTT), the vanishing and exploding gradient problems, and the reason why gated architectures such as LSTM and GRU are useful. A second part of the experiment extended the same idea to videos. Here, a CNN was used to obtain features from individual frames and a recurrent network was used to model the sequence of frame features.

The UCI Human Activity Recognition Using Smartphones dataset was used for the main sequence-classification experiment. Finally, a small sequence-to-sequence reversal problem was implemented to get a basic understanding of encoder–decoder models.

2 Learning Outcomes

After completing the experiment, I was able to:

- represent sequential data in the (N, T, F) format;
- understand the basic working of a Vanilla RNN;
- understand how BPTT works and why gradients can vanish or explode;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare the three recurrent models using accuracy, precision, recall and F1-score;
- read training curves and confusion matrices;
- understand how a CNN can be used to obtain useful features from video frames;
- build a CNN–LSTM/CNN–GRU pipeline for a small video classification task;
- understand the encoder–decoder idea used in sequence-to-sequence learning; and
- compare the models by considering both their performance and their computational cost.

3 Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI HAR dataset contains smartphone sensor readings collected while people performed six different activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

For this experiment, the raw inertial signals were used instead of directly using the precomputed 561-feature representation. Each sample contains 128 time steps. The nine channels used were the three body-acceleration channels, the three gyroscope channels and the three total-acceleration channels.

Therefore, the input was arranged as

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset is available from:
https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones
To keep the experiment manageable, a balanced subset of 3000 windows was selected. Each activity contributed 500 windows. The same preprocessing and evaluation procedure was then used for the three recurrent models so that their results could be compared fairly.

Expected Input

For a single sample,

$$X_i \in \mathbb{R}^{128\times 9}.$$

For a batch of 32 samples,

$$X_{\text{batch}} \in \mathbb{R}^{32\times 128\times 9}.$$

Here, N represents the number of sequences, T is the number of time steps and F is the number of sensor features at each time step.

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

This three-dimensional representation is important because the recurrent layers need to know not only the feature values but also their order in time.

4 Overall Experimental Pipeline

The sequence classification procedure used in the experiment can be summarized as follows:



The same input data, output classes, optimizer, batch size and evaluation procedure were used for RNN, LSTM and GRU. Only the recurrent layer was changed.

5 Preprocessing

The following steps were carried out before training:

1. The raw inertial signals were loaded.
2. The signals were arranged into windows of size 128×9.
3. One activity label was assigned to every window.
4. The six activity labels were encoded.
5. The input values were normalized using statistics obtained from the training data.
6. The data was separated into training, validation and testing sets.
7. The class distribution was checked to make sure that the selected subset remained balanced.

The split used was

$$70\% \text{ training}, \qquad 15\% \text{ validation}, \qquad 15\% \text{ testing}.$$

The test set was kept separate from model selection and hyperparameter tuning.

Output Obtained

Balanced selection of 3000 windows was done, taking 500 samples for each category. With the 70/15/15 partition, there were 350 training samples, 75 validation samples, and 75 test samples for each category.

```
Input tensor shape:
Training   :  (2100, 128, 9)
Validation :  (450, 128, 9)
Testing    :  (450, 128, 9)
```

```
Number of classes:  6
Number of features per time step:  9
Sequence length:  128
```

6 Temporal Data Visualization

To see how the sensor values change with time, representative samples from different activities were plotted. Three channels were selected: body acceleration in the X direction, body gyroscope in the X direction and total acceleration in the X direction.

Plot 1: Sensor signal versus time

- X-axis: time step from 1 to 128;
- Y-axis: sensor value;
- three representative channels were plotted; and
- samples from different activity classes were compared.

Plot 1: Sensor signal vs. time (representative window per activity)

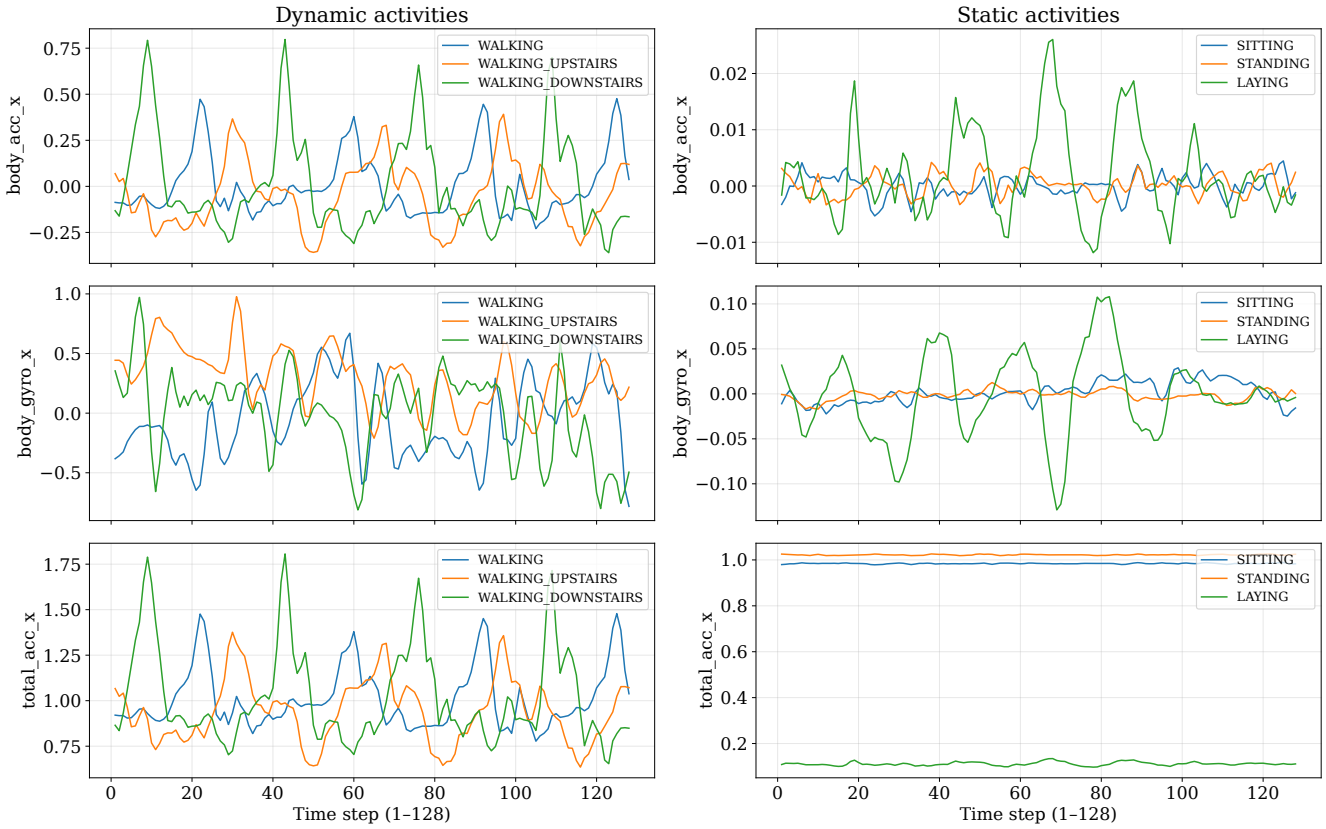


Figure 1: Sensor signals over 128 time steps for representative activity windows.

Inference: The activities related to walking exhibit greater and more consistent variations compared to sitting, standing, and laying. Walking data sets exhibit standard deviations of approximately 0.23 to 0.37 whereas the other low activity types are closer to 0.01. The three walking data sets, WALKING, WALKING_UPSTAIRS, and WALKING_DOWNSTAIRS, are also very similar to each other, which is why these classes are hard to distinguish. In an additional experiment, when the time order was randomly shuffled, validation accuracy dropped by 29.5 ppin case of LSTM and 21.1 ppin case of GRU.

7 Vanilla RNN Architecture

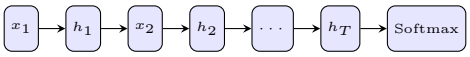
A Vanilla RNN processes the input one time step at a time. At every step it combines the current input with the hidden state obtained from the previous step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be represented as

$$y_t = g(W_y h_t + b_y).$$

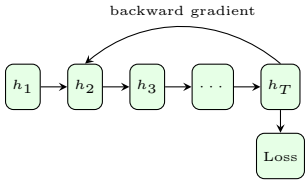
For sequence classification, the final hidden representation can be connected to a classifier.



The diagram is the unrolled view of the RNN. The important point is that the same recurrent weights are reused at every time step.

8 Backpropagation Through Time

During training, the recurrent network is effectively unfolded over all the time steps. The error is then propagated backward through these steps. This process is called Backpropagation Through Time (BPTT).



For distant time steps, the gradient contains repeated Jacobian terms. In a simplified form,

$$\frac{\partial h_T}{\partial h_{T-k}}$$

is a product of k such terms. If their magnitudes are less than one, the product can become extremely small. If their magnitudes are greater than one, it can grow very quickly. This gives rise to the vanishing-gradient and exploding-gradient problems.

In the numerical analysis carried out here, a spectral radius of 0.5 gives a value around 10^{-33} after 99 repeated multiplications, while a radius of 3 gives approximately 1.2×10^{13}. For the trained models, the ratio between the gradients at the initial and final time steps was approximately 1.4×10^{-8} for the RNN, 10^{-2} for the LSTM and 9×10^{-2} for the GRU.

Challenges

The main problems associated with a plain RNN are:

- vanishing gradients;
- exploding gradients;
- difficulty remembering information from much earlier time steps; and
- sequential computation, which limits parallelism.

The vanishing-gradient situation can be written conceptually as

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \to 0.$$

Exploding gradients occur when the magnitude of this quantity becomes very large.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

the hidden states are:

$$\begin{aligned}
h_1 &= \tanh(0.5 \cdot 0.5 + 0.8 \cdot 0 + 0.1) = \tanh(0.35) = 0.336376, \\
h_2 &= \tanh(0.5 \cdot 0.7 + 0.8 \cdot 0.336376 + 0.1) = \tanh(0.719100) = 0.616352, \\
h_3 &= \tanh(0.5 \cdot 0.2 + 0.8 \cdot 0.616352 + 0.1) = \tanh(0.693081) = 0.599958.
\end{aligned}$$

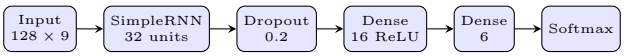
The values obtained manually were also checked using Keras SimpleRNN.

t	h_t (manual)	h_t (program)	Difference
1	0.336376	0.336376	4.0×10^{-8}
2	0.616352	0.616352	5.9×10^{-8}
3	0.599958	0.599958	2.7×10^{-8}

The small differences are due to floating-point precision.

9 RNN Implementation

The RNN model used in the experiment was intentionally kept small:



Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10 LSTM Architecture

The LSTM was used to address the difficulty of keeping useful information for a longer period of time. It maintains both a hidden state and a cell state. The gates decide which information should be removed, added and exposed.
The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

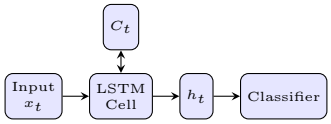
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C).$$

The cell state is updated as

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

Finally,

$$h_t = o_t \tanh(C_t).$$



The additive cell-state update is one of the main reasons the LSTM can preserve information more effectively than a basic RNN. The forget gate controls how much of the previous cell state is retained, while the input gate controls the new information that is written.

11 GRU Architecture

The GRU uses a simpler gated structure. It has an update gate and a reset gate:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z),$$

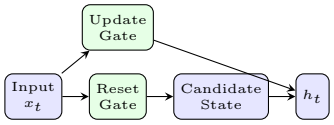
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The final hidden state is

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



In this experiment, the RNN had 1,974 trainable parameters, the LSTM had 6,006 and the GRU had 4,758. Thus, the GRU used fewer parameters than the LSTM while giving almost the same classification performance.

12 LSTM and GRU Implementation

The classifier after the recurrent layer was kept the same for all three models.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

The purpose of keeping the remaining parts of the architecture unchanged was to make the comparison between the recurrent cells as fair as possible.

13 Training Results

For each model, training and validation loss and accuracy were recorded.

Plot 2: Training and Validation Loss
Plot 3: Training and Validation Accuracy

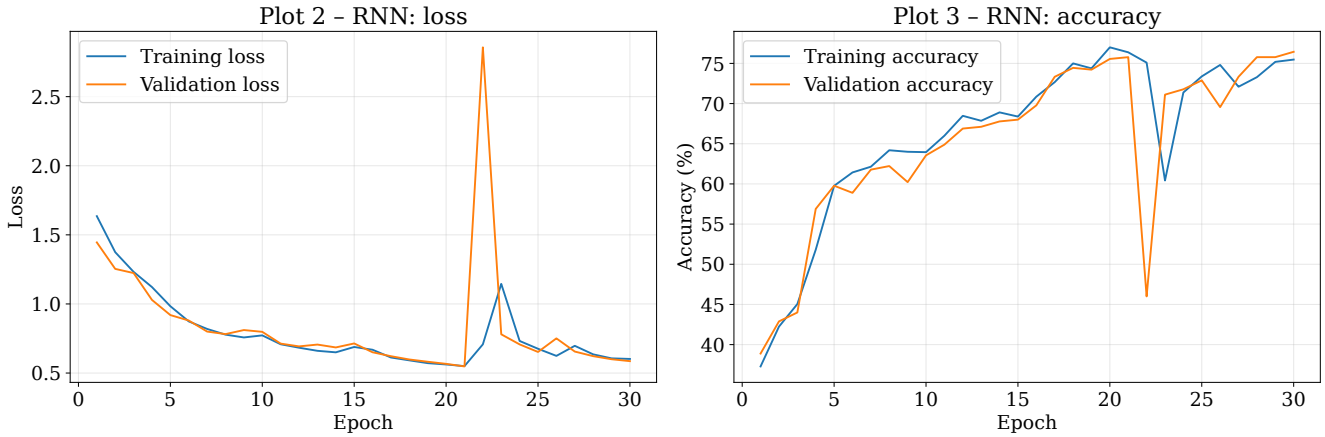


Figure 2: Training and validation curves for the RNN.

Inference: Learning for the RNN was quite slow. It managed to achieve a loss of about 0.60, while the accuracy hovered at mid-70%. There was also another visible transition during epochs 22–23. The validation accuracy achieved at the end is about 76%, showing that the basic RNN failed to harness the 128 steps sequence efficiently. The accuracies of both training and validation datasets were similar, so the problem was mainly underfitting.

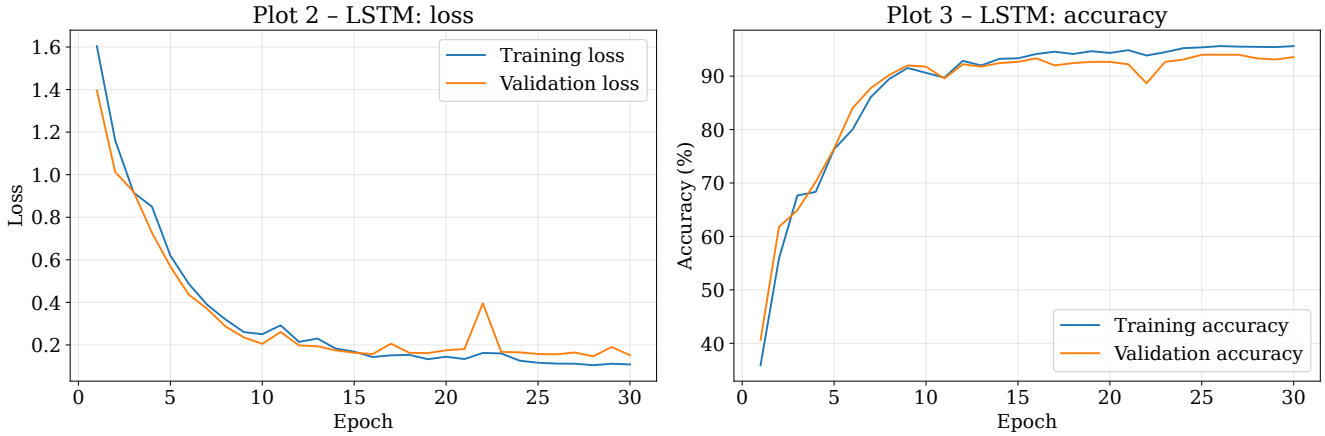


Figure 3: Training and validation curves for the LSTM.

Inference: LSTM showed better convergence results compared to the simple RNN. The validation accuracy got to 90% during epoch 8 and achieved 93.6% towards the end of the training process. Training accuracy was about 95.6%. The gap of 2.06 ppis rather insignificant, which means that overfitting is not likely to happen.

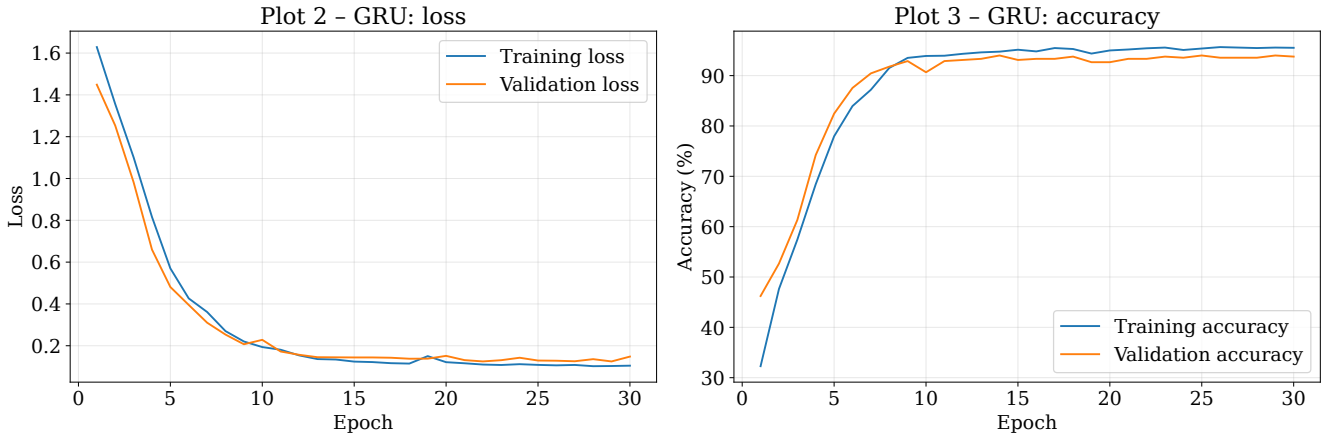


Figure 4: Training and validation curves for the GRU.

Inference: The GRU converged rapidly as well, surpassing the 90% validation accuracy point at epoch 7, ending up at 93.8% accuracy, with training reaching a bit over 95.5%. There was a mere difference of 1.75 percentage points between training and validation accuracy in this experiment.

14 Performance Evaluation

The trained models were evaluated on the test set, which was not used during training or model selection. Accuracy, macro precision, macro recall, macro F1-score, parameter count and training time were recorded.

Metric	RNN	LSTM	GRU
Accuracy (%)	78.67	95.33	95.11
Macro Precision (%)	78.95	95.41	95.27
Macro Recall (%)	78.67	95.33	95.11
Macro F1 (%)	78.40	95.33	95.09
Parameters	1,974	6,006	4,758
Training Time (s)	29.22	25.02	24.53

The gated models clearly performed better than the plain RNN. LSTM obtained the highest accuracy and F1-score, although the difference from GRU was very small.

15 Confusion Matrix Analysis

Separate confusion matrices were generated for all three recurrent models.

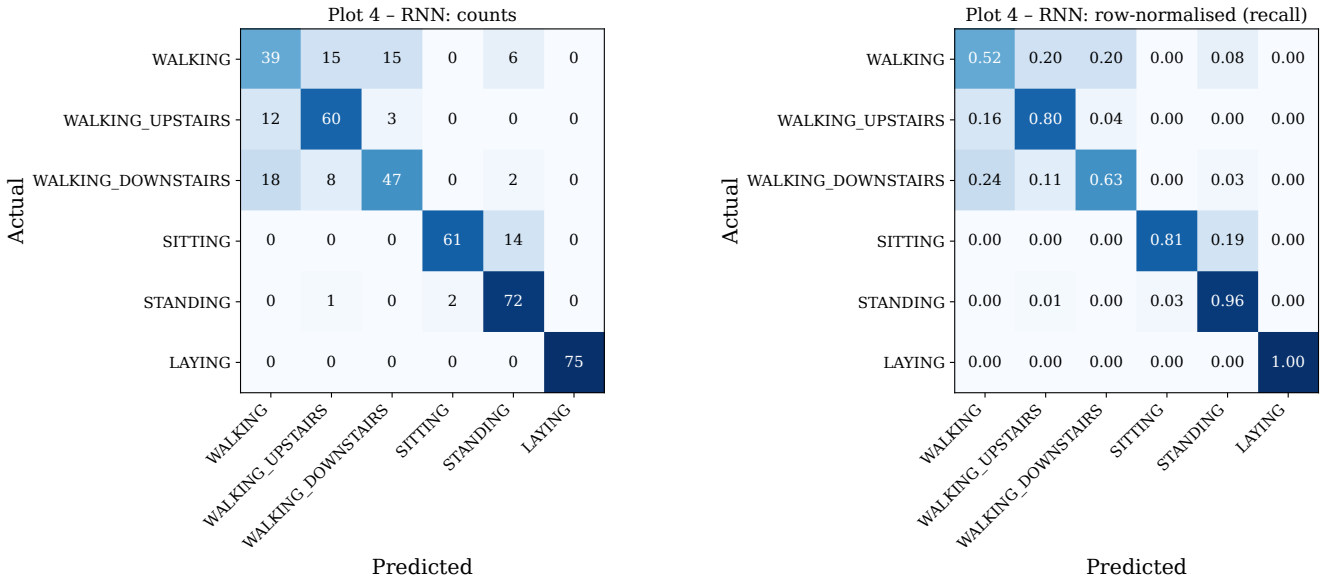


Figure 5: Confusion matrix for the RNN.

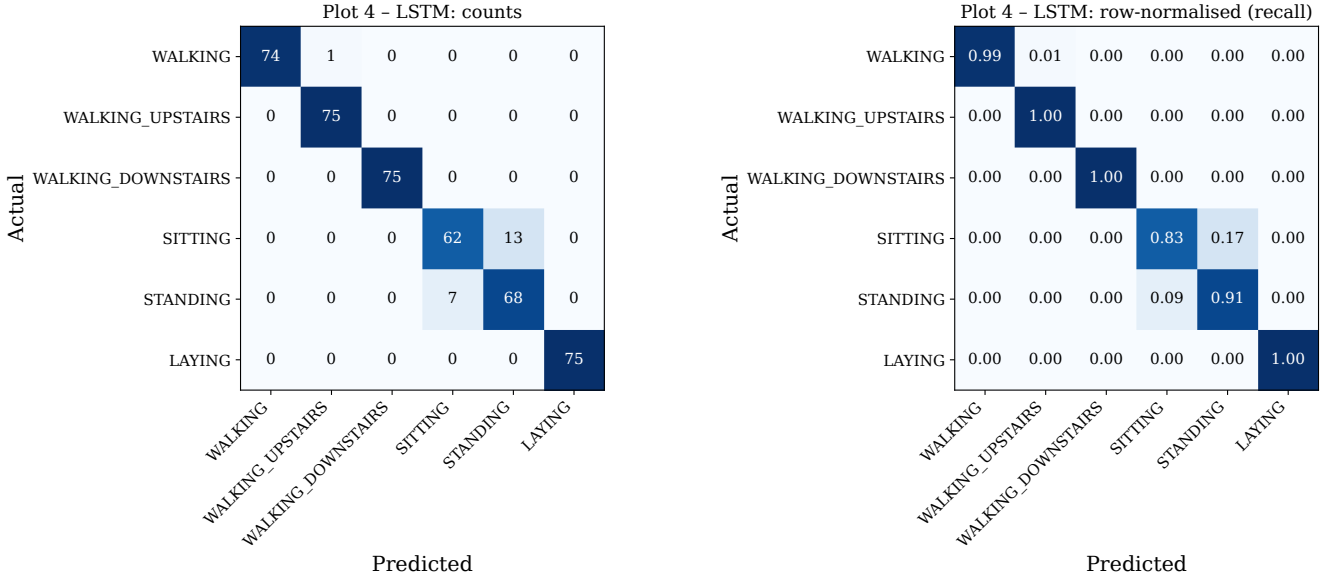


Figure 6: Confusion matrix for the LSTM.

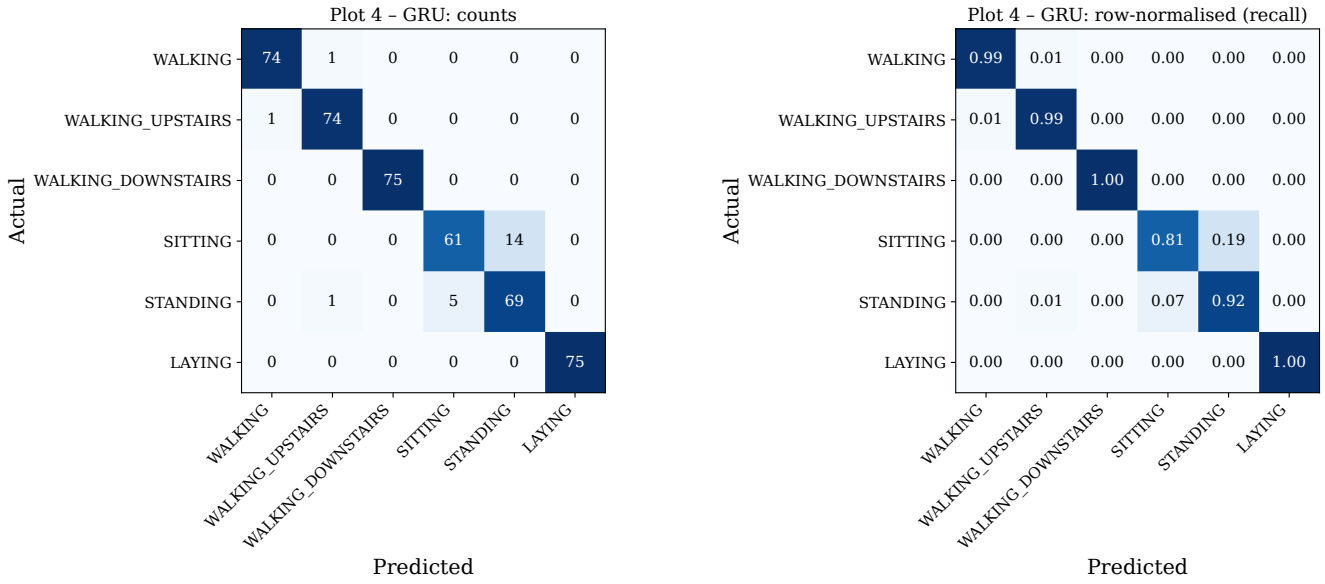


Figure 7: Confusion matrix for the GRU.

Inference: LAYING was detected accurately by all three methods. Recall of WALKING activity was lowest among all the activities for the RNN, at around 52%, and the problem was that the model confused WALKING with other activities of walking nature. LSTM and GRU performed much better with these categories. However, their biggest challenge was distinguishing between SITTING and STANDING activities, and that is logical since the sensor data may look very similar.

16 RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time	29.22 s	25.02 s	24.53 s
Test F1-score	78.40%	95.33%	95.09%

Plot 5: Model Performance Comparison

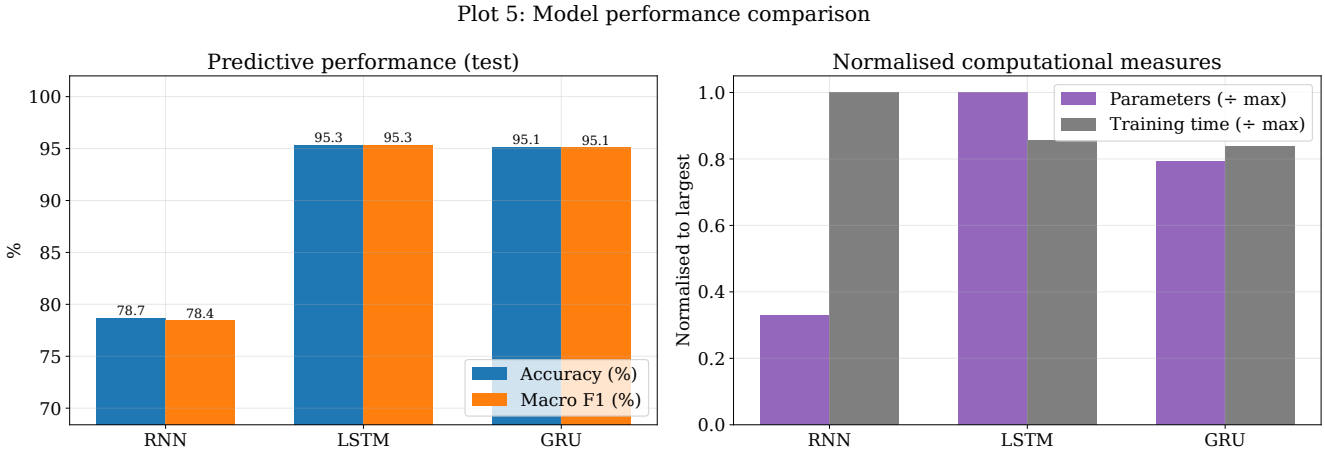


Figure 8: Comparison of the three recurrent models.

Inference: The key difference lies in the performance. While LSTM and GRU achieved around 95%, the RNN could achieve just 78.7%. In terms of number of parameters, the LSTM model has the highest number of parameters compared to GRU and RNN. Interestingly, RNN took the longest time in this particular execution, despite having the least number of parameters. Hence, parameter count does not have any impact on the training time. With regards to the two models, GRU is preferred since it achieves similar performance using fewer parameters and less training time.

17 Effect of Sequence Length

To see how much the amount of temporal information affected the models, the experiment was repeated with sequence lengths of

$$T \in \{32, 64, 128\}.$$

The resulting macro F1-scores were:

Sequence Length	RNN F1	LSTM F1	GRU F1
32	75.95	93.09	93.53
64	85.35	92.03	94.20
128	78.40	95.33	95.09

Plot 6: Sequence Length vs. Test F1-score

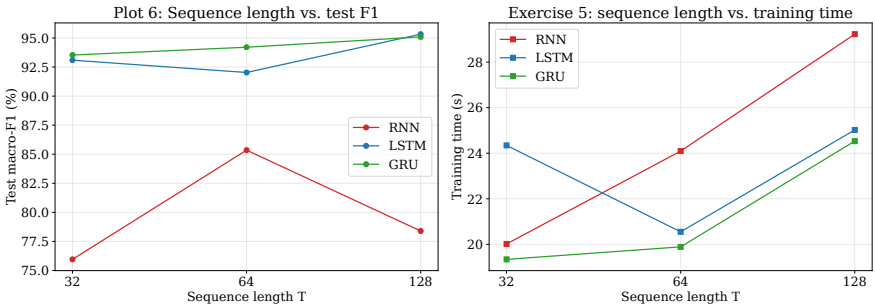


Figure 9: Effect of sequence length on test macro F1-score.

Inference: The extension of sequence length aided both the LSTM and GRU models. The LSTM performance increased from 93.09% accuracy at 32 time steps to 95.33% accuracy at 128 time steps, while the GRU accuracy rose from 93.53% to 95.09%. However, the RNN did not benefit much from an increased sequence length; it achieved accuracies of 75.95%, 85.35% and 78.40%.

18 Video Understanding Using CNN and RNN

The second part of the experiment applied the same sequence-learning idea to videos. A video can be viewed as a sequence of image frames. A CNN can process each frame and extract spatial information, while an RNN, LSTM or GRU can process the sequence of extracted features.

The pipeline used was:



Dataset

A small subset of the UCF101 action-recognition dataset was used. The selected classes were Basketball, Biking, WalkingWithDog, TennisSwing and JumpRope. There were 30 training videos, 10 validation videos and 10 test videos per class.

The UCF101 dataset is available from:
https://www.crcv.ucf.edu/data/UCF101.php

The goal was not to build a state-of-the-art video classifier. The purpose was mainly to understand how frame-level CNN features can be arranged into a temporal sequence.

19 Expected Video Input and Output

Ten frames were sampled uniformly from every video. Each frame was resized to

$$224 \times 224 \times 3.$$

MobileNetV2 was used as a pretrained CNN feature extractor. If the extracted feature dimension is D, one video becomes

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of videos,

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The recurrent classifier then produces probabilities for the five selected action classes.
For example:

```
Predicted class :  Basketball
Actual class    :  Basketball
Confidence      :  0.xx
```

The confidence value was produced by the trained classifier.

20 CNN Feature Extraction

MobileNetV2 was used without its final classification head.

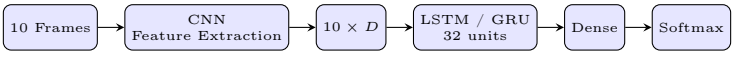


The CNN was frozen and used only for feature extraction. This reduced the amount of computation required for the small video dataset.

Observation: For the frozen MobileNetV2, a feature vector of dimension 1280 was extracted from each frame. So the representation of one video is $(10, 1280)$. The total dimensions of the feature tensors were $(150, 10, 1280)$, $(50, 10, 1280)$, and $(50, 10, 1280)$ respectively.

21 CNN–LSTM / CNN–GRU Model

The extracted frame features were given to an LSTM or GRU with 32 units.



Plot 7: Video Sample Frames



Figure 10: Ten uniformly sampled frames from a Basketball video.

Inference: The CNN produces a compressed version of all that appears in the image, which includes objects, scenes and even the visual appearance of the player. This is then fed into the recurrent layer. Given that the CNN was pretrained before, training the recurrent classifier was sufficient on the small number of videos.

Plot 8: Video Training and Validation Curves

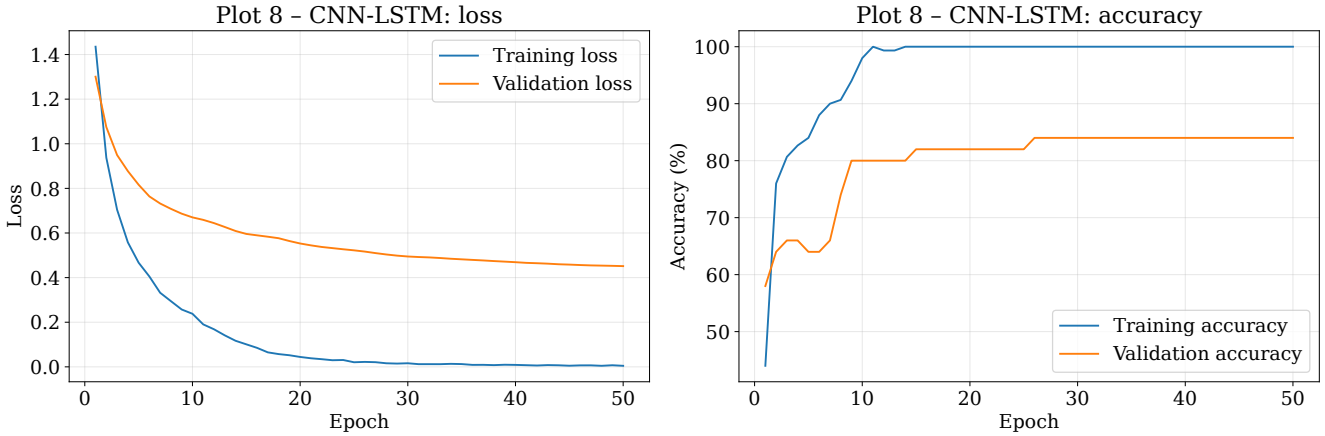


Figure 11: Training and validation curves for the CNN–LSTM model.

The CNN–LSTM model was selected because its test accuracy was 84.0%, compared with 78.0% for CNN–GRU.

Inference: The video model trained on the training set had high accuracy, close to 100%. Accuracy in the validation set remained in the 80–84% range, showing a big discrepancy between the two sets. This implies that there is overfitting since there is an almost 16 ppdisparity. It is important to note that only 200 videos were available for training.

Plot 9: Video Confusion Matrix

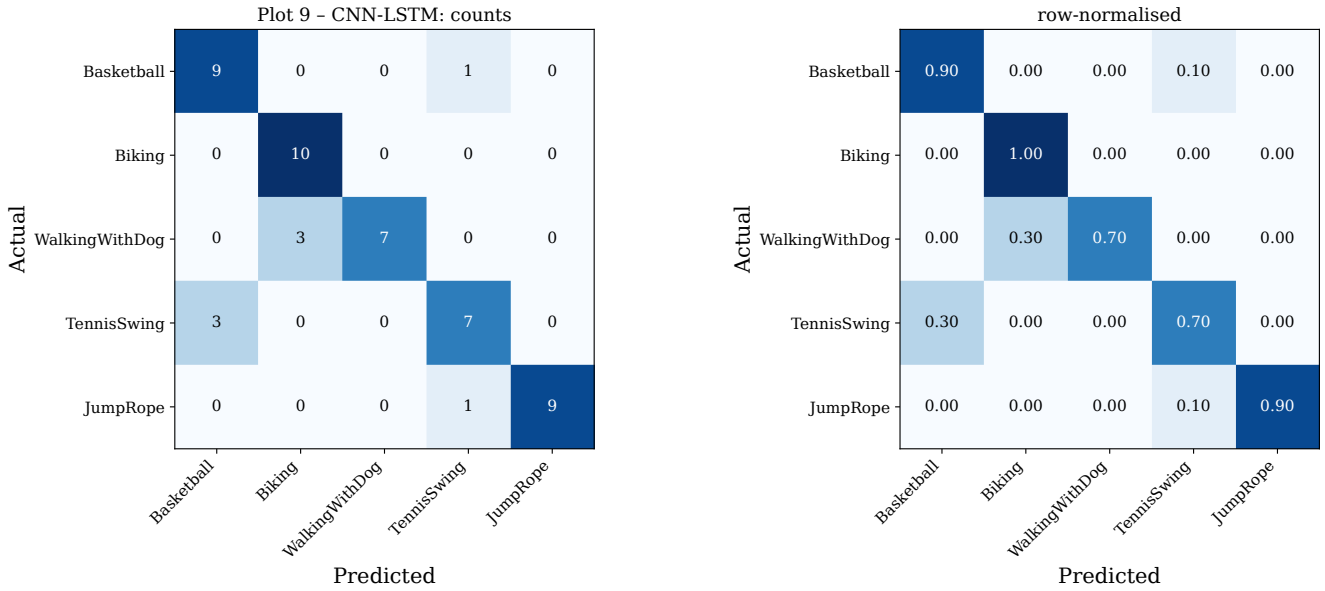


Figure 12: CNN–LSTM confusion matrix on the 50 test videos.

Inference: Biking was identified accurately in all 10 test videos. Basketball and JumpRope both had 9 correct classifications. WalkingWithDog had 7 correct classifications, and three samples were classified as Biking. Similarly, TennisSwing had 7 correct classifications, and three samples were classified as Basketball. This is justified because there are some similarities in visual patterns among some of the classes. Changing the order of frames randomly did not help in improving the accuracy. The accuracy went down from 84.0% to 83.4%.

22 Sequence-to-Sequence Learning

Sequence-to-sequence models are useful when both the input and output are sequences. For this experiment, a simple sequence-reversal problem was used so that the encoder and decoder could be studied without introducing another large dataset.

For example,

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Several thousand short sequences were generated using integers from a fixed range.



23 Expected Sequence-to-Sequence Input and Output

One example is

$$[3, 8, 1, 5] \rightarrow [5, 1, 8, 3].$$

Five test examples obtained from the model are shown below.

Sample	Input Sequence	Predicted Output
1	[7, 5, 7, 8, 4]	[4, 8, 7, 5, 7]
2	[2, 4, 6, 2, 7]	[7, 2, 6, 4, 2]
3	[2, 3, 1, 9, 9]	[9, 9, 1, 3, 2]
4	[7, 9, 6, 9, 8]	[8, 9, 6, 9, 7]
5	[3, 3, 6, 3, 3]	[3, 3, 6, 3, 3]

24 Sequence-to-Sequence Evaluation

Two accuracy measures were considered. Token accuracy measures individual positions, while sequence accuracy requires the entire predicted sequence to be correct.

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

For the 1800 test sequences, both token accuracy and sequence accuracy were 100.00%. The training loss was 2.17×10^{-5} and the validation loss was 3.28×10^{-5}.

Inference: This particular task was fairly easy for the model to solve, so both token accuracy and sequence accuracy were at 100%. However, for a more challenging sequence task, these two figures may differ from each other greatly. Assuming that token level accuracy is p and that there is independence among errors, we can roughly estimate the accuracy of a sequence of length L by p^L. For instance, $0.98^5 \approx 90.4\%$.

25 Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	78.67	78.95	78.67	78.40	1,974
LSTM	95.33	95.41	95.33	95.33	6,006
GRU	95.11	95.27	95.11	95.09	4,758
CNN–LSTM/GRU	84.00	85.94	84.00	83.91	168,677

All classification metrics are percentages and are macro-averaged. The last row corresponds to the selected CNN–LSTM model. The CNN–GRU model obtained 78.00% accuracy, 80.11% precision, 78.00% recall and 78.14% F1-score with 126,757 parameters.

For the sequence-to-sequence experiment:

Task	Token Acc. (%)	Seq. Acc. (%)	Train Loss	Val. Loss	Parameters
Sequence reversal	100.00	100.00	2.17×10^{-5}	3.28×10^{-5}	150,090

26 Required Inferences

1. **Temporal pattern.** Walking actions are significantly more periodic compared to non-walking actions in terms of sensor data signal. Moreover, the similarity in the features of walking classes is responsible for some of the classification mistakes made. Shuffling the temporal order of data significantly worsened the validation accuracy results.
2. **RNN convergence.** The RNN model converged significantly slower compared to other two models. The accuracy was at around 76% while the loss function was at about 0.60. This result corresponds well to the complexity of gradient backpropagation through 128 steps.
3. **LSTM convergence.** The LSTM network was able to reach 90% validation accuracy relatively early in training and finished at 93.6%. Gated state in the cell layer allowed the model to keep information during training.
4. **GRU convergence.** The GRU network reached 90% validation accuracy by about epoch 7 and achieved final accuracy of 93.8%. This result is very close to the LSTM accuracy despite the fact that GRU uses less parameters.
5. **Overfitting or underfitting.** It appears that the RNN underfitted since its training and validation accuracy was roughly around the mid-70% range. For LSTM and GRU, the gap was relatively small. Conversely, CNN–LSTM video model clearly had overfitting problem since its training accuracy is close to 100% while the validation accuracy is only around 84%.
6. **Activity-wise errors.** Walking activity was hard to recognize for the RNN while Laying activity was correctly recognized. For LSTM and GRU models, the source of confusion was Sitting versus Standing activities.
7. **Sequence length.** Longer sequences tended to work better for the gated models. Specifically, LSTM improved from 93.09% F1 score at 32 steps to 95.33% F1 score at 128 steps. Similarly, GRU improved from 93.53% to 95.09%. No such trend could be observed for the RNN.
8. **Parameters and computation.** Three models used 1,974, 6,006 and 4,758 parameters respectively. Although the RNN used least amount of parameters, it was quite slower in this case taking 29.22 seconds, while LSTM and GRU took 25.02 and 24.53 seconds respectively.

9. **Role of the CNN.** With the frozen MobileNetV2, the images of dimensionality $224 \times 224 \times 3$ were converted into vectors of dimensionality 1280, allowing the recurrent network to deal with compressed frame vectors rather than the images themselves.
10. **Role of the recurrent model.** The role of the recurrent layer was to find out the dynamics of frame features over time. In the current experiment, there was not much benefit from temporal order information, which could be partially caused by the small amount of data.
11. **Token versus sequence accuracy.** Token and sequence accuracies are both equal to 100% for the simplest reverse problem. However, sequence accuracy is more strict since even one wrong token means an incorrect sequence.

27 Discussion Questions

1. What is a recurrent neural network?
   Answer: An RNN is a neural network specialized to process sequential data. Its inputs are processed one step at a time with information carried from previous steps in the hidden state:
   $$h_t = f(x_t, h_{t-1}).$$
2. What does the hidden state contain?
   Answer: The hidden state is the learned representation of the information observed by the network so far. It is a kind of memory.
3. Why is order of elements in sequence important?
   Answer: Same set of numbers can be interpreted differently depending on the order. In this experiment, the random rearrangement of the sequence of sensor readings reduced validation accuracy drastically for LSTM and GRU.
4. What is the difference between a feedforward network and an RNN?
   Answer: A feedforward network does not carry any state between its inputs while RNN does.
5. Backpropagation Through Time
   Answer: In BPTT, the network gets unfolded according to its time steps, and the error gradient is backpropagated through the time steps. The gradients of the recurrent weight parameters are summed throughout the whole sequence.
6. How can vanishing gradients happen in BPTT?
   Answer: The error gradient consists of a product of many Jacobian matrices. When these terms have small magnitude values (i.e., less than one), their consecutive multiplications make the gradient even smaller. In the end, the early time step contributions become negligible for the update.
7. How can exploding gradients be observed in BPTT?
   Answer: If the magnitudes of the Jacobian matrices exceed one, the error gradient becomes extremely large. To control such situations, gradient clipping is a popular technique.
8. What are long-term dependencies?
   Answer: Long-term dependencies refer to the case where the current output depends on some data that has appeared in previous time steps. It may be hard to learn them by simple RNNs due to the problem of vanishing gradients.
9. Why do we need the LSTM cell state?
   Answer: The cell state gives us extra memory channel along the sequence. It is updated via the use of the forget and input gates:
   $$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$
10. What are the forget, input and output gates in LSTM?
    Answer: The forget gate regulates which information is to be forgotten from the previous cell state. The input gate regulates what new information is added and the output gate regulates what part of the cell state is revealed.
11. What are the reset and update gates in GRU?
    Answer: The reset gate regulates what part of the previous hidden state affects the candidate state. The update gate regulates the balance between the previous state and the new one.
12. How are LSTM and GRU similar?
    Answer: LSTM has its own cell state with three primary gates while GRU has memory function embedded into its hidden state with only two gates. In our experiment, the number of parameters was 6,006 for LSTM and 4,758 for GRU. Also, their F1-scores were very close.
13. Why does GRU use a simpler gating system than LSTM?
    Answer: This is because GRU does not have a separate cell state and uses two gates as opposed to the three main gates of LSTM.

14. Why should RNN, LSTM and GRU be compared using the same conditions?
Answer: If this is not done, differences in the data split, pre-processing or training will influence the outcome. This will make it easier to attribute the performance difference to the architecture of the neural network.
15. What does a confusion matrix reveal that accuracy does not?
Answer: Accuracy provides one value whereas the confusion matrix reveals the confusion between the classes. In our experiment, this revealed that the SITTING and STANDING classes still posed problems despite the accuracy being about 95% in LSTM/GRU.
16. Why should macro F1-score be used in multi-class classification?
Answer: Because it treats every class as equally important. This way, it ensures that performance on one class does not overshadow poor performance on another class.
17. What is the definition of sequence length?
Answer: Sequence length is the number of time steps allotted to the recurrent model. Longer sequence means more temporal information to the model, while longer recurrent computation.
18. Why is a CNN able to extract features from video?
Answer: Videos are composed of image frames and CNN are capable of extracting spatial features from images. Therefore, a pretrained CNN can be utilized independently to each frame.
19. What type of information does the CNN extract?
Answer: The CNN is able to extract spatial information in form of shapes, textures, objects, scenes and visual patterns from each image frame.
20. What type of information does the recurrent model learn?
Answer: The recurrent model receives as its input a sequence of CNN features and learns how these features evolve over time.
21. Why should we create a sequence of CNN features before applying LSTM/GRU?
Answer: Each video consists of several frames and CNN extracts one feature vector per frame. Thus, these vectors are sorted in temporal order and form an input of size (B, T, F) for the recurrent layer.
22. Explain the Encoder and Decoder in Sequence to Sequence Learning.
Answer: The Encoder takes in the input sequence and creates a representation of it. The Decoder uses the representation created by the Encoder and generates the output sequence.
23. What is teacher forcing?
Answer: While training the model, teacher forcing is used to give the correct previous target token to the Decoder instead of the previous predicted token generated by the Decoder itself. Teacher forcing simplifies the training process.
24. Explain token accuracy and sequence accuracy.
Answer: Token Accuracy validates the accuracy of each individual predicted element. Sequence Accuracy validates the whole predicted sequence.
25. Why should test data never be used in model selection?
Answer: The test data set is used for getting the final unbiased evaluation of the model performance. If the test data is used multiple times in model selection, the performance obtained from it will be inflated.

28 Expected Outcome

The experiment demonstrated the complete workflow for sequence classification:

$$\text{Sequential Data} \rightarrow \text{Preprocessing} \rightarrow \text{RNN/LSTM/GRU} \rightarrow \text{Evaluation}.$$

The video experiment extended the same idea:

$$\text{Video} \rightarrow \text{Frames} \rightarrow \text{CNN Features} \rightarrow \text{LSTM/GRU} \rightarrow \text{Action Prediction}.$$

The final part introduced the encoder–decoder structure:

$$\text{Input Sequence} \rightarrow \text{Encoder} \rightarrow \text{Context} \rightarrow \text{Decoder} \rightarrow \text{Output Sequence}.$$

The key observation from the results is that LSTM and GRU are considerably better compared to a plain RNN in the case of 128-step sensor sequence. The GRU model is particularly interesting as it gave a similar result as the LSTM model but had fewer parameters. In addition, the video experiment indicated that CNN models can be used together with recurrent models to take into account both spatial and temporal aspects.

29 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: `https://www.tensorflow.org`
7. Keras Documentation: `https://keras.io`